

Learning language-model RL from an official reproduction

We want to establish that our language-model reinforcement learning pipeline does what we think it does: generate answers, assign rewards, change weights, and evaluate those changed weights fairly. We start with the authors’ code for *Understanding R1-Zero-Like Training: A Critical Perspective* [1,2]. This is a small reproduction of its training method, with a smaller model workload and budget. It does not reproduce the paper’s published scores.

Why do this before recursive language-model RL?

In this experiment, the language model receives a mathematics question and writes one response. A checker compares its final answer with a reference. There are no live Python tools or recursive model calls in these responses. That makes the reward and update easier to inspect.

An RLM adds choices about decomposition, running code, reading intermediate results, and calling other models. A failure can then come from the language model, the surrounding software, the reward, or which actions receive credit. A working ordinary LLM experiment gives us a reference for the shared training machinery. It does not prove that those additional RLM decisions are trained correctly. We will still need separate checks of action boundaries, tool results, costs, and credit assignment.

Pretraining, SFT, and RL are different sources of learning

Stage	What supplies the learning target?
Pretraining	Predicting text from a large existing collection develops the starting language and mathematical abilities.
Supervised fine-tuning (SFT)	Supplied example responses tell the model which tokens to imitate for a prompt.
Reinforcement learning (RL)	The model samples its own responses. Rewards change how likely those responses are under its policy.

Our starting checkpoint is the **Qwen2.5-Math-1.5B base model** [3]. We add no SFT stage. “Base” does not mean untrained: its weights already contain substantial capabilities from earlier training. Calling this R1-Zero-like training does not mean beginning with random weights.

What has been established so far?

Training with the chat prompt improved its full-500 score from **154 to 308**. But the base model already scored **305 with the question alone**. On question-only evaluation, the chat-trained model scored **317**; two fresh question-only training runs scored **314 and 306**. Against 305, their gains of nine and one answer do not demonstrate a stable, material improvement. We have validated a working RL pipeline and strong prompt sensitivity; dependable learning gains remain uncertain. Pages 4–5 explain the comparisons, and page 6 examines individual answers from the pilot.

Evidence cutoff: 7 October 2026, 12:35 UTC. Three training runs evaluated in both formats; further evaluation repeat pending.

How eight answers produce an update

For each training question, the generator samples eight responses at temperature 1. The author reward function extracts the last boxed answer and checks it against the reference. A correct answer earns 1; a wrong or unparseable answer earns 0. There is no separate reward just for using a box. The training actor also assigns 0 to a response stopped by the token limit. The checker evaluates the answer, not the truth of every reasoning step [2].

A teaching example: two successes in a group of eight

Suppose one question produces rewards

$$r = (1, 0, 0, 1, 0, 0, 0, 0), \quad \bar{r} = 2/8 = 0.25.$$

Dr. GRPO subtracts this group’s mean from each reward:

$$A_i = r_i - \bar{r}, \quad A = (0.75, -0.25, -0.25, 0.75, -0.25, -0.25, -0.25, -0.25).$$

The two successful responses receive positive advantages; the other six receive negative advantages. The update pushes toward the successful sampled responses and away from the unsuccessful ones. If all eight answers are wrong, or all eight are correct, their advantages are all zero. The sum being zero in a mixed group does not cancel the learning signal: different responses have different token probabilities and gradients.

What is special about Dr. GRPO?

Dr. GRPO is the authors’ variant of Group Relative Policy Optimization. It does not divide advantages by the group’s reward standard deviation. It also sums token losses and divides by a constant response limit, here 3000, instead of dividing each response’s loss by its own length. These are the method’s two normalization changes [1,2]. Its policy loss uses a clipped probability ratio:

$$q_{it} = \frac{\pi_{\theta}(y_{it} \mid x_i, y_{i,<t})}{\pi_{\text{old}}(y_{it} \mid x_i, y_{i,<t})}, \quad L = -\frac{1}{B \cdot 3000} \sum_{i=1}^B \sum_{t \in \text{response}_i} \min(q_{it} A_i, \text{clip}(q_{it}, 0.8, 1.2) A_i).$$

Here the policy is the model’s next-token distribution, θ is its set of weights, and $B = 128$ responses. The group advantage is shared across all response tokens; this experiment does not determine which individual reasoning step deserves credit. We use zero KL penalty: no extra penalty keeps the updated policy close to a reference model.

How the single-GPU run fits together

One collection contains 16 questions times eight responses: 128 trajectories. The learner processes one trajectory at a time, accumulates 128 gradients, and takes one optimizer update. It then sends the new weights to vLLM for the next collection. We use one pass over each collection, full-model BF16 training, learning rate 10^{-6} , and the author’s Adam settings.

The pilot’s 64 questions gave four updates; each completed 512-question run gave 32 updates from 4096 sampled responses. Training and generation alternate on one A100 with 40 GB of memory. Putting the generator to sleep frees memory; it does not reset the weights. Likewise, switching a model between training and evaluation modes changes its execution behavior, not its learned weights. The optimizer performs the update.

Freeze the questions and the evaluator

We use the official code at commit `dfca49dd460e`, Oat 0.1.3.post1, vLLM 0.8.4, and Qwen model revision `4a83ca6e4526`. Full identifiers, package versions, and content hashes are recorded beside this guide in the research record and data manifest [4].

Purpose	Size	Selection and use
Pilot training	64	First 64 indices after shuffling 12,000 author training rows with Python <code>random.Random(42)</code> .
Larger training	512	First 512 indices from the same shuffle. This includes the pilot questions; every larger run restarts from the base weights.
Monitoring	64	Shuffle the author’s 500 MATH evaluation rows with seed 142; take the first 64. Reused for progress checks.
Final comparison	128	Take the next 128 from that evaluation shuffle. Disjoint from monitoring; used for the larger run’s fixed final comparison.

The 12,000 training rows include 4,501 IDs from the original MATH test split. An arbitrary download of that test split would therefore be a poor choice of evaluation data. We checked the author’s supplied training collection against the supplied MATH500 questions and found no exact overlaps after whitespace normalization. This does not rule out paraphrases or exposure during the base model’s pretraining.

Keep three kinds of settings separate

Weights. The baseline uses the original base checkpoint. Each trained model uses its prescribed final checkpoint. All three 512-question runs start separately from the base, not from the pilot or each other’s weights.

Generation. Each before/after pair uses the same input format, the same 3000-token response limit, and greedy decoding with one answer per question. We evaluate both the Qwen-Math chat prompt and the question alone; page 5 shows what changes. Training uses temperature 1 for varied samples. Comparing base and trained weights under different prompts would mix prompt effects with weight changes. This is a central warning of the paper [1].

Scoring. The author’s training program uses a faster checker. The standalone evaluator uses an additional mathematical-equivalence check (`fast=False`). We apply the same standalone checker to both saved answer sets for every reported comparison, including both formats for the newly completed question-trained model. Source questions and reference answers were checked against the frozen data. We never combine scores from different checkers.

Choose the checkpoint before reading the final score

We retain the prescribed final checkpoint and evaluation, even if an earlier evaluation looks better. On the 64 monitoring questions, the chat-trained run’s final evaluation scored 36, versus 20 at baseline; another evaluation after the same 32 updates scored 39. The pilot likewise scored 23 and then 22 with identical saved weights. Greedy GPU decoding was not exactly repeatable. We report this variation without assigning it a specific numerical cause or choosing the higher score.

The initial runs use learner seed 42 and the fresh question-only repeat uses 43. The unchanged author actor chooses a time-based generation seed, so learner seed labels do not make the whole run deterministic. Evaluation repeats test generation variation; fresh training runs test whether a gain survives retraining.

A large prompt effect, an uncertain learning gain

Three separate runs restarted from the original base weights. Each used the same 512 questions, 4096 sampled responses, and 32 full-model BF16 updates in the unchanged author algorithm. One used the chat prompt and learner seed 42; two used questions alone, with learner seeds 42 and 43. These are separate training runs, not multiple evaluations of one trained model [4].

Evaluation input	Starting	Chat-trained seed 42	Question-trained seed 42	Question-trained seed 43
Chat prompt	154/500	308/500	168/500	166/500
Question alone	305/500	317/500	314/500	306/500

Within each row, models are compared under matched evaluation settings. Between rows, the input format changes. All trained columns use prescribed final checkpoints. The question-only follow-ups were chosen after seeing the stronger question-only baseline, so they are exploratory, not new untouched confirmation.

On question-only evaluation, chat-prompt training gains 12 answers (18 gains, six losses). The first question-only training run gains nine (24 gains, 15 losses), and the fresh repeat gains one (18 gains, 17 losses). Both differences are positive, but +1.8 and +0.2 percentage points do not establish a stable, material improvement. The repeat improves nine answers and worsens 17 relative to the first question-trained model, for eight fewer correct answers overall. Under chat evaluation it scores 166, with 25 gains and 13 losses against 154.

What do the different gains mean?

The large gain remains the chat-trained model under chat evaluation, from 30.8% to 61.6%. Against the stronger question-only baseline, the fresh training repeat weakens the case for a dependable benefit from this short recipe. This neither proves zero learning nor identifies newly acquired mathematics. It also does not establish that all gains are answer-format repair, or validate RLM tool use and credit assignment across multiple calls.

The chat-trained model reduced capped chat responses from 205 to 29; the two question-trained models still had 201 and 198 in that format. With question-only evaluation, the base had 15 capped responses, the first question-trained model 19, and its fresh repeat 14. Their mean lengths were 694.31, 693.61, and 680.15 tokens. The repeat produced no empty answers. These descriptive measurements do not explain the cause of each changed score.

Keep the original fixed test intact

For the chat-trained model, the separate prescribed 128-question comparison remains **41/128 before** and **80/128 after**, with 44 gains and five losses. The newly generated full-500 batch contains those same 128 questions, but its chat-format counts are 41 and 82. That later observation does not replace the original result. Changing evaluation batches can change generated answers.

In the original comparison, capped responses fell from 56 to 12. Of its 44 gains, 36 had a capped baseline response and 37 had no parseable baseline answer. Those categories overlap. They identify completion as an important limitation, but cannot distinguish changes in reasoning, repetition, and answer formatting on their own.

A chat prompt is extra input, not new weights

The official `qwen_math` option wraps each question in these role markers and adds a system instruction. The placeholder below stands for the unchanged dataset question [2].

```
<|im_start|>system
Please reason step by step, and put your final answer within \boxed{<|im_end|>
<|im_start|>user
[question]<|im_end|>
<|im_start|>assistant
```

The official `no` option passes just the question text. It adds no role markers and no instruction about reasoning or boxing the answer. Both formats still use the same model architecture, greedy decoding, 3000-token response limit, and final-answer checker. The question-only baseline demonstrates that this pretrained base model can already answer many questions without those extra instructions. Only the base model’s input format has changed.

Which questions make up the full-500 result?

All 500 questions were excluded from our weight training. Their roles in this study nevertheless differ: 64 were repeatedly monitored, 128 formed the original fixed test, and 308 add further coverage. These counts come from the full-500 evaluations, with each trained checkpoint kept separate:

Input	Question role	Starting	Chat seed 42	Questions seed 42	Questions seed 43
Question only	Monitored 64	39	37	39	37
	Original test 128	79	84	84	81
	Additional 308	187	196	191	188
Chat prompt	Monitored 64	21	42	24	23
	Original test 128	41	82	45	40
	Additional 308	92	184	99	103

The additional questions broaden the evaluation, but the full set is not 500 entirely uninspected questions. More evaluation rows are not more training replicates. By the fresh training repeat, all 500 had already been evaluated. We do not know whether these problems appeared during the base model’s pretraining.

Check the actual inputs, then repeat the result

The 10:35 question-only attempt was rejected: despite its command-line setting, a dataset cache supplied chat-formatted training inputs. The valid fresh run used an isolated cache, started at 10:40, and finished by 11:24. All 4096 saved training prompts were checked against the 512 raw source questions. A setting in a configuration file is not evidence of what the model actually received.

The unchanged-weight evaluations repeated exactly, including the saved answers: 305 for the base and 314 for the first question-trained model. A fresh question-only training run scored 306. A further evaluation of that saved model is planned; no result from it is included here. All completed results are retained. Re-evaluation tests generation; retraining tests learning. The repeat’s final monitoring score was 41/64, but the same 64 rows scored 37 in its full-500 batch. That generation difference does not change the reported 306/500 or justify selecting the better batch.

What did the pilot’s two extra points mean?

Under the standalone checker, 19 monitoring answers stayed correct, 41 stayed incorrect, three became correct, and one became incorrect. Thus the net change is $+2/64$, or 3.125 percentage points. These are the earlier pilot’s **monitoring results**, separate from the larger run’s 41-to-80 test result.

Changed problem	What the saved responses show
Fuel use	Repetition without an answer became a completed calculation and boxed 550.
Magic square	The baseline already derived $n = 7$, then repeated an incorrect verification. The final response completed and boxed 7.
Recurrence	A boxed answer changed from 1089 to the correct 898. Both responses had valid answer formatting.
Trigonometry	A correct boxed 12 became repetitive noncompletion at the 3000-token limit.

A concrete example: getting an answer out reliably

Karla drove 12,000 miles. How much fuel would she save with a car averaging 48 miles per gallon instead of 15? The completed post-training response gives

$$\frac{12000}{15} = 800, \quad \frac{12000}{48} = 250, \quad 800 - 250 = \boxed{550}.$$

The baseline repeated a version of the question until the 3000-token limit. The final response completed in approximately 417 text tokens. This is useful behavioral improvement on that example, but it does not show that the model newly acquired division or subtraction.

The recurrence example is also instructive. Independent calculation gives $211 + 420 + 267 = 898$. Both responses wrote Python describing the correct recurrence, but the baseline’s claimed code output was 1089. Those fenced “output” blocks were generated text; no Python tool had run. A correct final answer does not validate an entire explanation or prove successful tool use.

What the training evidence adds

The first collection had 12 rewarded responses out of 128, with seven mixed groups giving 56 nonzero advantages. Twenty responses reached the token limit. The four completed optimizer updates had finite nonzero gradients, sampled checkpoint tensors changed, and updated weights were sent back to the generator. This establishes an operational path from sampled answers to weight updates. It does not make the small monitoring gain statistically conclusive.

The pilot illustrates why an aggregate score needs accompanying examples. The larger chat-format gain and the modest question-only gains both need the same care in interpretation.

Sources and editable evidence.

- [1] Liu et al., Understanding R1-Zero-Like Training: A Critical Perspective. <https://arxiv.org/abs/2503.20783>.
- [2] Pinned official training, evaluation, and grading code; Oat package version.
- [3] Pinned Qwen2.5-Math-1.5B model revision.
- [4] Companion research record, data manifest, and resolved environment; main training receipt; fixed-128 scoring receipt; full-500 chat-format receipt; prompt-comparison receipt; question-only training receipt; question-trained final scores; fresh-repeat training receipt; fresh-repeat final scores; pilot receipt. The research record links the question-only answer files and scoring evidence.